

AULA 01

Objetivos

- Entender **MVC** usando um projeto real: **Interclasse**
- Identificar claramente o papel de:
 - **Model** (dados + regras + banco)
 - **View** (telas)
 - **Controller** (fluxo/coordenação)
- Implementar partes do sistema: **Jogadores, Modalidades, Partidas, Placar e Cartões**
- Organizar rotas e pastas como um projeto “de verdade” (sem framework)

1) Contexto do projeto (domínio)

No sistema interclasse, temos objetos do mundo real

(entidades): • **Jogador** (nome, turma, número, etc.)

• **Modalidade** (futsal, vôlei, basquete...)

• **Time** (turma A vs turma B, por modalidade)

• **Partida** (data, times, modalidade)

• **Placar** (gols/pontos por time na partida)

• **Cartão** (amarelo/vermelho, jogador, minuto, partida)

*Gancho didático: MVC não é sobre “telas bonitas”, é sobre **organizar responsabilidades** para conseguir crescer o sistema sem sofrimento.*

2) O problema que MVC resolve (no Interclasse)

Sem MVC, um arquivo partida.php vira “tudo ao mesmo

tempo”: • lê \$_POST

- valida regra (ex.: “cartão vermelho expulsa”)
- faz INSERT e SELECT
- imprime HTML
- calcula placar e classificação

Com o tempo, você tem medo até de corrigir um ;.

Com MVC:

- **Model:** sabe buscar/gravar e aplicar regras
- **Controller:** coordena a ação (“cadastrar partida”, “lançar placar”, “registrar cartão”)
- **View:** só renderiza (exibe) os dados

3) Conceito de MVC (traduzido para o Interclasse)

Model (Modelo)

“Quem entende do campeonato e do banco”

- Ex.: Partida::create(), Cartao::registrar(), Jogador::allByTurma() •

Faz SQL via **PDO**

- Regras de negócio (mínimas, mas reais):
 - Não criar partida sem modalidade
 - Não registrar cartão para jogador que não está na partida (opcional no início)
 - Placar não pode ser negativo

View (Visão)

“As telas do sistema”

- Ex.: listar partidas, formulário de nova partida, tela de placar e cartões

- HTML + foreach + if
- **Sem SQL** e sem “regras pesadas”

Controller (Controle)

“O maestro”

- Recebe requisição (rota + GET/POST)
- Chama Models
- Escolhe a View certa
- Faz redirect após POST (PRG: Post/Redirect/Get)

4) Estrutura de pastas (MVC “caseiro” e didático) text

Copiar

interclasse/

public/

index.php

app/

controllers/

PartidaController.php

JogadorController.php

ModalidadeController.php

CartaoController.php

models/

Database.php

Jogador.php

Modalidade.php

Partida.php

Cartao.php

views/

partidas/

index.php

create.php

show.php

jogadores/

index.php

modalidades/

index.php

sql/

schema.sql

Ideia-chave: public/index.php é a porta de entrada (Front

Controller). **5) Banco de dados (MySQL) — modelo inicial do**

Interclasse Abaixo um **schema mínimo** (dá para evoluir depois):

sql

Copiar

CREATE DATABASE interclasse CHARACTER SET utf8mb4 COLLATE

utf8mb4_unicode_ci;

USE interclasse;

4

Programação para Web III Introdução ao MVC

```
CREATE TABLE modalidades (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nome VARCHAR(80) NOT NULL UNIQUE  
);
```

```
CREATE TABLE jogadores (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nome VARCHAR(120) NOT NULL,  
    turma VARCHAR(20) NOT NULL,  
    numero INT NULL  
);
```

```
CREATE TABLE partidas (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    modalidade_id INT NOT NULL,  
    data_hora DATETIME NOT NULL,  
    time_casa VARCHAR(20) NOT NULL,  
    time_fora VARCHAR(20) NOT NULL,  
    placar_casa INT NOT NULL DEFAULT 0,  
    placar_fora INT NOT NULL DEFAULT 0,
```

```
FOREIGN KEY (modalidade_id) REFERENCES modalidades(id)

);
```

5

Programação para Web III Introdução ao MVC

```
CREATE TABLE cartoes (

id INT AUTO_INCREMENT PRIMARY KEY,

partida_id INT NOT NULL,

jogador_id INT NOT NULL,

tipo ENUM('amarelo','vermelho') NOT NULL,

minuto INT NULL,

observacao VARCHAR(255) NULL,

FOREIGN KEY (partida_id) REFERENCES partidas(id),

FOREIGN KEY (jogador_id) REFERENCES jogadores(id)

);
```

Decisão didática proposital: placar fica em partidas para simplificar no começo. Depois dá para normalizar.

6) Models (com PDO) — exemplos com cara de Interclasse 6.1

Database.php (conexão)

app/models/Database.php

php

Copiar

<?php

class Database

```
{  
  
private static ?PDO $conn = null;
```

6

Programação para Web III Introdução ao MVC

```
public static function connection(): PDO  
  
{  
  
if (self::$conn === null) {  
  
$dsn = "mysql:host=localhost;dbname=interclasse;charset=utf8mb4";  
  
self::$conn = new PDO($dsn, "root", "", [  
  
PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,  
  
PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC ]);  
  
}  
  
return self::$conn;  
  
}  
  
}
```

6.2 Model de Partida

app/models/Partida.php

php

Copiar

<?php

```
require_once __DIR__ . '/Database.php';
```

```
class Partida
```

```
{
```

```
public static function all(): array
```

7

Programação para Web III Introdução ao MVC

```
{
```

```
$pdo = Database::connection();
```

```
$sql = "SELECT p.*, m.nome AS modalidade
```

```
FROM partidas p
```

```
JOIN modalidades m ON m.id = p.modalidade_id ORDER
```

```
BY p.data_hora DESC";
```

```
return $pdo->query($sql)->fetchAll();
```

```
}
```

```
public static function find(int $id): ?array
```

```
{
```

```
$pdo = Database::connection();
```

```
$stmt = $pdo->prepare(
```

```
"SELECT p.*, m.nome AS modalidade
```

```
FROM partidas p
```

```
JOIN modalidades m ON m.id = p.modalidade_id
```

```
WHERE p.id = :id"
```

```
);
```

```
$stmt->execute(['id' => $id]);
```

```

$row = $stmt->fetch();

return $row ? : null;

}

```

8

Programação para Web III Introdução ao MVC

```

public static function create(int $modalidadeId, string $dataHora, string $casa,
string $fora): int

{

    // regra simples (poderia evoluir depois)

    if ($casa === $fora) {

        throw new InvalidArgumentException("Times não podem ser iguais."); }

    $pdo = Database::connection();

    $stmt = $pdo->prepare(

        "INSERT INTO partidas (modalidade_id, data_hora, time_casa, time_fora)

        VALUES (:mid, :dh, :casa, :fora)"

    );

    $stmt->execute([

        'mid' => $modalidadeId,

        'dh' => $dataHora,

        'casa' => $casa,

        'fora' => $fora

    ]);

```

```
return (int)$pdo->lastInsertId();  
  
}
```

9

Programação para Web III Introdução ao MVC

```
public static function atualizarPlacar(int $partidaid, int $casa, int $fora): void  
{  
  
    if ($casa < 0 || $fora < 0) {  
  
        throw new InvalidArgumentException("Placar não pode ser negativo."); }  
  
    $pdo = Database::connection();  
  
    $stmt = $pdo->prepare(  
  
        "UPDATE partidas  
  
        SET placar_casa = :casa, placar_fora = :fora  
  
        WHERE id = :id"  
  
    );  
  
    $stmt->execute(['casa' => $casa, 'fora' => $fora, 'id' => $partidaid]); }  
  
}
```

6.3 Model de Cartão

app/models/Cartao.php

php

Copiar

<?php

```
require_once __DIR__ . '/Database.php';
```

40

Programação para Web III Introdução ao MVC

```
class Cartao
{
    public static function allByPartida(int $partidaId): array
    {
        $pdo = Database::connection();
        $stmt = $pdo->prepare(
            "SELECT c.*, j.nome AS jogador, j.turma
            FROM cartoes c
            JOIN jogadores j ON j.id = c.jogador_id
            WHERE c.partida_id = :pid
            ORDER BY c.id DESC"
        );
        $stmt->execute(['pid' => $partidaId]);
        return $stmt->fetchAll();
    }

    public static function registrar(int $partidaId, int $jogadorId, string $tipo, ?int
    $minuto, ?string $obs): int
    {
        if (!in_array($tipo, ['amarelo', 'vermelho'], true)) {
```

```
throw new InvalidArgumentException("Tipo de cartão inválido."); }
```

44

Programação para Web III Introdução ao MVC

```
$pdo = Database::connection();

$stmt = $pdo->prepare(

    "INSERT INTO cartoes (partida_id, jogador_id, tipo, minuto, observacao)

    VALUES (:pid, :jid, :tipo, :minuto, :obs)"

);

$stmt->execute([

    'pid' => $partidaId,

    'jid' => $jogadorId,

    'tipo' => $tipo,

    'minuto' => $minuto,

    'obs' => $obs

]);

return (int)$pdo->lastInsertId();

}

}
```

7) Controllers — “casos de uso” do Interclasse

O que o Controller faz aqui?

Ele traduz ações do sistema em métodos:

- PartidaController@index → listar partidas

- PartidaController@createForm → mostrar formulário
- PartidaController@store → salvar partida
- PartidaController@show → detalhes (placar + cartões)

42

Programação para Web III Introdução ao MVC

- PartidaController@updateScore → atualizar placar (POST)

app/controllers/PartidaController.php

php

Copiar

<?php

```
require_once __DIR__ . '/../models/Partida.php';
require_once __DIR__ . '/../models/Cartao.php';
require_once __DIR__ . '/../models/Database.php';
```

```
class PartidaController
```

```
{
```

```
    public function index(): void
```

```
    {
```

```
        $partidas = Partida::all();
```

```
        require __DIR__ . '/../views/partidas/index.php'; }
    }
```

```
    public function show(): void
```

```
    {
```

```
        $id = (int)($_GET['id'] ?? 0);
```

```
$partida = Partida::find($id);
```

43

Programação para Web III Introdução ao MVC

```
if (!$partida) {
```

```
    http_response_code(404);
```

```
    echo "Partida não encontrada.";
```

```
    return;
```

```
}
```

```
$cartoes = Cartao::allByPartida($id);
```

```
require __DIR__ . '/../views/partidas/show.php';
```

```
}
```

```
public function createForm(): void
```

```
{
```

```
    // Carregar modalidades para o <select>
```

```
    $pdo = Database::connection();
```

```
    $modalidades = $pdo->query("SELECT id, nome FROM modalidades ORDER  
    BY nome")->fetchAll();
```

```
require __DIR__ . '/../views/partidas/create.php';
```

```
}
```

```
public function store(): void
```

```
{
```

14

Programação para Web III Introdução ao MVC

```
$modalidadeId = (int)($_POST['modalidade_id'] ?? 0);
```

```
$dataHora = trim($_POST['data_hora'] ?? "");
```

```
$casa = trim($_POST['time_casa'] ?? "");
```

```
$fora = trim($_POST['time_fora'] ?? "");
```

```
if ($modalidadeId <= 0 || $dataHora === "" || $casa === "" || $fora === "") {
```

```
    http_response_code(422);
```

```
    echo "Dados inválidos para criar partida.";
```

```
    return;
```

```
}
```

```
try {
```

```
    Partida::create($modalidadeId, $dataHora, $casa, $fora);
```

```
    header("Location: /partidas");
```

```
    exit;
```

```
} catch (Throwable $e) {
```

```
    http_response_code(400);
```

```
    echo $e->getMessage();
```

```
}
```

```
}
```

```
public function updateScore(): void
```

```
{
```

15

Programação para Web III Introdução ao MVC

```
$id = (int)($_POST['id'] ?? 0);
```

```
$casa = (int)($_POST['placar_casa'] ?? -1);
```

```
$fora = (int)($_POST['placar_fora'] ?? -1);
```

```
try {
```

```
Partida::atualizarPlacar($id, $casa, $fora);
```

```
header("Location: /partidas/show?id=" . urlencode((string)$id)); exit;
```

```
} catch (Throwable $e) {
```

```
http_response_code(400);
```

```
echo $e->getMessage();
```

```
}
```

```
}
```

```
}
```

*Repare no padrão **PRG**: após POST, o Controller dá header("Location: ...") para evitar reenviar formulário ao dar F5.*

8) Views — telas com cara de sistema escolar

8.1 Listagem de partidas

app/views/partidas/index.php

php

Copiar

```
<!doctype html>
```

```
<html lang="pt-br">
```

46

Programação para Web III Introdução ao MVC

```
<head>
```

```
<meta charset="utf-8">
```

```
<title>Partidas - Interclasse</title>
```

```
</head>
```

```
<body>
```

```
<h1>Partidas</h1>
```

```
<p><a href="/partidas/create">Cadastrar nova partida</a></p>
```

```
<table border="1" cellpadding="6">
```

```
<thead>
```

```
<tr>
```

```
<th>Data</th>
```

```
<th>Modalidade</th>
```

```
<th>Confronto</th>
```

```
<th>Placar</th>
```

```
<th>Ações</th>
```

```
</tr>
```

```
</thead>
```

```
<tbody>
```

```
<?php foreach ($partidas as $p): ?>
```

```
<tr>
```

```
<td><?= htmlspecialchars($p['data_hora']) ?></td>
```

```
<td><?= htmlspecialchars($p['modalidade']) ?></td>
```

17

Programação para Web III Introdução ao MVC

```
<td><?= htmlspecialchars($p['time_casa']) ?> x <?=
htmlspecialchars($p['time_fora']) ?></td>
```

```
<td><?= (int)$p['placar_casa'] ?> : <?= (int)$p['placar_fora'] ?></td>
```

```
<td>
```

```
<a href="/partidas/show?id=<?= urlencode((string)$p['id']) ?>">Detalhes</a>
```

```
</td>
```

```
</tr>
```

```
<?php endforeach; ?>
```

```
</tbody>
```

```
</table>
```

```
</body>
```

```
</html>
```

8.2 Detalhe da partida (placar + cartões)

app/views/partidas/show.php

php

Copiar

```
<!doctype html>
```

```

<html lang="pt-br">

<head>

<meta charset="utf-8">

<title>Detalhes da Partida</title>

</head>

```

48

Programação para Web III Introdução ao MVC

```

<body>

<p><a href="/partidas">← Voltar</a></p>

```

```

<h1><?= htmlspecialchars($partida['modalidade']) ?> — <?=
htmlspecialchars($partida['time_casa']) ?> x <?=
htmlspecialchars($partida['time_fora']) ?></h1>

```

```

<p><strong>Data:</strong> <?= htmlspecialchars($partida['data_hora'])
?></p>

```

```

<h2>Placar</h2>

```

```

<form method="post" action="/partidas/placar">

```

```

<input type="hidden" name="id" value="<?= (int)$partida['id'] ?>">

```

```

Casa: <input type="number" name="placar_casa" value="<?=
(int)$partida['placar_casa'] ?>" min="0">

```

```

Fora: <input type="number" name="placar_fora" value="<?=
(int)$partida['placar_fora'] ?>" min="0">

```

```

<button type="submit">Atualizar</button>

```

```

</form>

```

```
<h2>Cartões</h2>
```

```
<ul>
```

```
<?php foreach ($cartoes as $c): ?>
```

```
<li>
```

```
<?= htmlspecialchars($c['tipo']) ?> — <?= htmlspecialchars($c['jogador']) ?>
```

49

Programação para Web III Introdução ao MVC

```
(<?= htmlspecialchars($c['turma']) ?>)
```

```
<?php if (!empty($c['minuto'])): ?>
```

```
— <?= (int)$c['minuto'] ?>'
```

```
<?php endif; ?>
```

```
<?php if (!empty($c['observacao'])): ?>
```

```
— <?= htmlspecialchars($c['observacao']) ?>
```

```
<?php endif; ?>
```

```
</li>
```

```
<?php endforeach; ?>
```

```
</ul>
```

```
</body>
```

```
</html>
```

Ponto de segurança: htmlspecialchars() protege contra XSS quando alguém “trolla” o sistema com texto malicioso.

9) Rotas (Front Controller) para o Interclasse

public/index.php (roteamento simples)

php

Copiar

<?php

```
require_once __DIR__ . '/../app/controllers/PartidaController.php';
```

```
$path = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);
```

```
$method = $_SERVER['REQUEST_METHOD'];
```

20

Programação para Web III Introdução ao MVC

```
$partidaController = new PartidaController();
```

```
if ($path === '/' || $path === '/partidas') {
```

```
    $partidaController->index();
```

```
} elseif ($path === '/partidas/show') {
```

```
    $partidaController->show();
```

```
} elseif ($path === '/partidas/create' && $method === 'GET')
```

```
{ $partidaController->createForm();
```

```
} elseif ($path === '/partidas/create' && $method === 'POST') {
```

```
    $partidaController->store();
```

```
} elseif ($path === '/partidas/placar' && $method === 'POST')
```

```
{ $partidaController->updateScore();
```

```
} else {
```

```
    http_response_code(404);
```

```
echo "Rota não encontrada.";
```

```
}
```

10) “Mapa” MVC do sistema interclasse (para fixação) A

turma costuma memorizar melhor assim:

Funcionalidade Listar partidas Criar partida

createForm/store

Partida::cr

Controller	Mod
PartidaController@index	Parti

24

Programação para Web III Introdução ao MVC

Controller	Model	View
show	Partida::find(), Cartao::allByPartida()	views/partidas/show.php
updateScore	Partida::atualizarPlacar()	(redirect para show)
CartaoController@store	Cartao::registrar()	(redirect para show)

Funcionalidade

Detalhes da
partida

Atualizar placar

Registrar cartão

11) Atividade prática (20–30 min) — “Cartão no jogo”

Objetivo: Registrar cartões na tela de detalhes da partida.

Passos: 1) Criar rota POST /cartoes/create 2) Criar CartaoController@store 3) Na view partidas/show.php, criar um formulário:

- jogador_id
- tipo (amarelo/vermelho)
- minuto (opcional)
- observação (opcional) 4) Após salvar, redirect de volta para /partidas/show?id=...

Correção esperada: cartão aparece na lista sem recarregar/duplicar submissão ao dar F5.

12) Extensões naturais (quando o sistema crescer)

- **Tabela de classificação** por modalidade (pontos, saldo, vitórias)
- **Time como tabela** (não só texto “turma”): permite elencos por modalidade
- **Relacionar jogador na partida** (escalação) para validar cartão com precisão

22

Programação para Web III Introdução ao MVC

- Camada extra: **Service** (quando regras ficarem mais complexas que CRUD)
- Segurança: **CSRF token** em formulários (quando virar “produção”)

AULA 02

Padronizando as telas com Bootstrap 5 (layout único)

Referência: [Bootstrap 5 \(docs\)](#) — [Bootstrap 5 \(docs\)](#)

A ideia “MVC raiz” mais limpa é:

- **Views de páginas** (ex.: views/partidas/index.php) contêm **só o conteúdo**

- Um **layout global** (ex.: views/layout.php) contém <html>, navbar, Bootstrap, etc.
- O Controller manda: **qual view renderizar + quais dados**

Estrutura (evoluída)

text

Copiar

interclasse/

public/

index.php

app/

controllers/

Controller.php

PartidaController.php

ModalidadeController.php

23

Programação para Web III Introdução ao MVC

JogadorController.php

CartaoController.php

models/

Database.php

Partida.php

Modalidade.php

Jogador.php

Cartao.php

views/

layout.php

partials/

flash.php

home/

index.php

partidas/

index.php

create.php

show.php

modalidades/

index.php

create.php

jogadores/

index.php

24

Programação para Web III Introdução ao MVC

create.php

sql/

schema.sql

2) Flash messages (mensagens de sucesso/erro) — “cara de sistema”

Vamos usar \$_SESSION para mostrar mensagens após redirects.

public/index.php (início do arquivo)

php

Copiar

<?php

session_start();

app/views/partials/flash.php

php

Copiar

<?php if (!empty(\$_SESSION['flash'])): **?>**

<?php \$f = \$_SESSION['flash']; unset(\$_SESSION['flash']); **?>**

<div *class*="alert alert-**<?=** htmlspecialchars(\$f['type']) **?>** alert-dismissible fade show" *role*="alert">

<?= htmlspecialchars(\$f['message']) **?>**

<button *type*="button" *class*="btn-close" *data-bs-dismiss*="alert" *aria-label*="Fechar">**</button>**

</div>

<?php endif; **?>**

25

Programação para Web III Introdução ao MVC

3) Layout Bootstrap (Navbar + Container)

app/views/layout.php

php

Copiar

<?php

```

// $title (string) opcional

// $view (string) obrigatório, ex.: 'partidas/index'

// $data (array) opcional

?>

<!doctype html>

<html lang="pt-br">

<head>

<meta charset="utf-8">

<meta name="viewport" content="width=device-width, initial-scale=1">

<title><?= htmlspecialchars($title ?? 'Interclasse') ?></title>


<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">

</head>

<body class="bg-light">


26
Programação para Web III Introdução ao MVC

<nav class="navbar navbar-expand-lg navbar-dark
bg-success"> <div class="container">

<a class="navbar-brand fw-bold" href="/">Interclasse</a>

<button class="navbar-toggler" type="button" data-bs-toggle="collapse" data
bs-target="#nav">

<span class="navbar-toggler-icon"></span>

```

</button>

<div class="collapse navbar-collapse" id="nav">

<ul class="navbar-nav ms-auto">

<li class="nav-item">Partidas

<li class="nav-item">Jogadores

<li class="nav-item"><a class="nav-link"
href="/modalidades">Modalidades

</div>

</div>

</nav>

<main class="container py-4">

<?php require __DIR__ . '/partials/flash.php'; ?>

<?php

27

Programação para Web III Introdução ao MVC

// disponibiliza \$data para a view

if (!empty(\$data)) extract(\$data, EXTR_SKIP);

require __DIR__ . '/' . \$view . '.php';

?>

</main>

```

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js
"></script>

</body>

</html>

```

4) Base Controller: render + redirect + flash (organiza MUITO)

app/controllers/Controller.php

php

Copiar

<?php

class Controller

{

protected function render(string \$view, array \$data = [], string \$title =
'Interclasse'): void

{

require __DIR__ . '/../views/layout.php';

28

Programação para Web III Introdução ao MVC

}

protected function redirect(string \$to, ?string \$flashMessage = null, string
\$flashType = 'success'): void

```

{
    if ($flashMessage !== null) {
        $_SESSION['flash'] = ['type' => $flashType, 'message' => $flashMessage];
    }

    header("Location: $to");

    exit;
}
}

```

Resultado: Controllers ficam menores e mais “profissionais”.

5) Models adicionais (Modalidade e Jogador)

app/models/Modalidade.php

php

Copiar

<?php

```
require_once __DIR__ . '/Database.php';
```

```
class Modalidade
```

```

{

    public static function all(): array

    {

        $pdo = Database::connection();
    }
}

```

```

return $pdo->query("SELECT id, nome FROM modalidades ORDER BY
nome")->fetchAll();

}

```

```

public static function create(string $nome): int
{
    $nome = trim($nome);

    if ($nome === "") throw new InvalidArgumentException("Nome da modalidade é
obrigatório.");

```

```

    $pdo = Database::connection();

    $stmt = $pdo->prepare("INSERT INTO modalidades (nome) VALUES
(:nome)");

    $stmt->execute(['nome' => $nome]);

    return (int)$pdo->lastInsertId();

}

}

```

app/models/Jogador.php

php

Copiar

<?php

```

require_once __DIR__ . '/Database.php';

```

```

class Jogador
{
    public static function all(?string $turma = null): array
    {
        $pdo = Database::connection();

        if ($turma) {
            $stmt = $pdo->prepare("SELECT * FROM jogadores WHERE turma = :t
            ORDER BY nome");

            $stmt->execute(['t' => $turma]);

            return $stmt->fetchAll();
        }

        return $pdo->query("SELECT * FROM jogadores ORDER BY turma,
        nome")->fetchAll();
    }

    public static function create(string $nome, string $turma, ?int $numero): int
    {

```

```
$nome = trim($nome);
```

```
$turma = trim($turma);
```

```

if ($nome === " || $turma === ") {

throw new InvalidArgumentException("Nome e turma são obrigatórios."); }


$pdo = Database::connection();

$stmt = $pdo->prepare(

"INSERT INTO jogadores (nome, turma, numero) VALUES (:nome, :turma,
:numero)"

);

$stmt->execute([

'nome' => $nome,

'turma' => $turma,

'numero' => $numero

]);

return (int)$pdo->lastInsertId();

}

}

```

6) Controllers novos: Modalidade e Jogador (CRUD básico: listar + criar)

app/controllers/ModalidadeController.php

php

Copiar

<?php

```
require_once __DIR__ . '/Controller.php';
```

```
require_once __DIR__ . '/../models/Modalidade.php';
```

```
class ModalidadeController extends Controller
```

```
{
```

```
    public function index(): void
```

```
    {
```

```
        $modalidades = Modalidade::all();
```

```
        $this->render('modalidades/index', compact('modalidades'), 'Modalidades'); }

    
```

```
    public function createForm(): void
```

```
    {
```

```
        $this->render('modalidades/create', [], 'Nova Modalidade'); }

    
```

```
    public function store(): void
```

```
    {
```

33

Programação para Web III Introdução ao MVC

```
try {
```

```
    Modalidade::create($_POST['nome'] ?? "");
```

```
    $this->redirect('/modalidades', 'Modalidade cadastrada com sucesso!'); }
```

```

catch (Throwable $e) {

    $_SESSION['flash'] = ['type' => 'danger', 'message' => $e->getMessage()];

    $this->redirect('/modalidades/create');

}

}

}

```

app/controllers/JogadorController.php

php

Copiar

<?php

```

require_once __DIR__ . '/Controller.php';

require_once __DIR__ . '/../models/Jogador.php';

```

```

class JogadorController extends Controller

```

```

{

    public function index(): void

    {

        $turma = trim($_GET['turma'] ?? '');

        $turma = $turma !== '' ? $turma : null;
    }
}

```

```

$jogadores = Jogador::all($turma);

$this->render('jogadores/index', compact('jogadores', 'turma'), 'Jogadores'); }

```

```

public function createForm(): void
{
    $this->render('jogadores/create', [], 'Novo Jogador');
}

public function store(): void
{
    try {
        $numero = ($_POST['numero'] ?? '') !== '' ? (int)$_POST['numero'] : null;
        Jogador::create($_POST['nome'] ?? '', $_POST['turma'] ?? '', $numero);
        $this->redirect('/jogadores', 'Jogador cadastrado com sucesso!'); } catch
        (Throwable $e) {

            $_SESSION['flash'] = ['type' => 'danger', 'message' => $e->getMessage()];

            $this->redirect('/jogadores/create');

        }
    }
}

```

7) Controller de Cartão (registrar cartões na partida)

app/controllers/CartaoController.php

php

Copiar

<?php

```
require_once __DIR__ . '/Controller.php';
```

```
require_once __DIR__ . '/../models/Cartao.php';
```

```
class CartaoController extends Controller
```

```
{
```

```
    public function store(): void
```

```
    {
```

```
        $partidaId = (int)($_POST['partida_id'] ?? 0);
```

```
        try {
```

```
            $minuto = ($_POST['minuto'] ?? "") !== "" ? (int)$_POST['minuto'] : null;
```

```
            Cartao::registrar(
```

```
                $partidaId,
```

```
                (int)($_POST['jogador_id'] ?? 0),
```

```
                (string)($_POST['tipo'] ?? ""),
```

```
                $minuto,
```

```
                trim($_POST['observacao'] ?? "") ? : null
```

```
            );
```

```

$this->redirect('/partidas/show?id=' . urlencode((string)$partidaId), 'Cartão
registrado!');

} catch (Throwable $e) {

$_SESSION['flash'] = ['type' => 'danger', 'message' => $e- >getMessage()];

$this->redirect('/partidas/show?id=' . urlencode((string)$partidaId)); }

}

}

```

8) Rotas (Front Controller) — sistema mais completo

public/index.php (roteamento expandido)

php

Copiar

<?php

```
session_start();
```

```
require_once __DIR__ . '/../app/controllers/PartidaController.php';
```

```
require_once __DIR__ .
```

```
'../app/controllers/ModalidadeController.php';
```

```
require_once __DIR__ . '/../app/controllers/JogadorController.php';
```

```
require_once __DIR__ . '/../app/controllers/CartaoController.php';
```

```

$path = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);

$method = $_SERVER['REQUEST_METHOD'];


$partidas = new PartidaController();

$modalidades = new ModalidadeController();

$jogadores = new JogadorController();

$cartoes = new CartaoController();


if ($path === '/') {

    // Home simples: redireciona para partidas por enquanto

    header("Location: /partidas");

    exit;

}


/** Partidas */

if ($path === '/partidas' && $method === 'GET') $partidas->index(); elseif

($path === '/partidas/show' && $method === 'GET') $partidas->show();

elseif ($path === '/partidas/create' && $method === 'GET') $partidas-
>createForm();

elseif ($path === '/partidas/create' && $method === 'POST') $partidas->store();


elseif ($path === '/partidas/placar' && $method === 'POST') $partidas-
>updateScore()

```

*/** Modalidades */*

```
elseif ($path === '/modalidades' && $method === 'GET') $modalidades->index();

elseif ($path === '/modalidades/create' && $method === 'GET') $modalidades-
>createForm();

elseif ($path === '/modalidades/create' && $method === 'POST')
$modalidades->store()
```

*/** Jogadores */*

```
elseif ($path === '/jogadores' && $method === 'GET') $jogadores->index();

elseif ($path === '/jogadores/create' && $method === 'GET') $jogadores-
>createForm();

elseif ($path === '/jogadores/create' && $method === 'POST') $jogadores-
>store()
```

*/** Cartões */*

```
elseif ($path === '/cartoes/create' && $method === 'POST') $cartoes->store();

else {

    http_response_code(404);

    echo "Rota não encontrada.";

}
```

9) Views com Bootstrap (bonitas e organizadas)

9.1 Modalidades: listagem

app/views/modalidades/index.php

php

Copiar

```
<div class="d-flex justify-content-between align-items-center  
mb-3"> <h1 class="h3 mb-0">Modalidades</h1>  
  
<a class="btn btn-success" href="/modalidades/create">Nova modalidade</a>  
  
</div>  
  
<div class="card shadow-sm">  
  
<div class="card-body">  
  
<ul class="list-group">  
  
<?php foreach ($modalidades as $m): ?>  
  
<li class="list-group-item d-flex justify-content-between align-items-center">  
  
<span><?= htmlspecialchars($m['nome']) ?></span>  
  
<span class="badge bg-secondary">#<?= (int)$m['id'] ?></span> </li>  
  
<?php endforeach; ?>  
  
</ul>  
  
</div>
```

</div>

9.2 Modalidades: cadastro

app/views/modalidades/create.php

php

Copiar

<h1 class="h3 mb-3">Nova modalidade</h1>

<div class="card shadow-sm">

<div class="card-body">

<form method="post" action="/modalidades/create" class="row g-3">

<div class="col-md-8">

<label class="form-label">Nome</label>

<input name="nome" class="form-control" placeholder="Ex.: Futsal, Vôlei..."
required>

</div>

<div class="col-12">

<button class="btn btn-success">Salvar</button>

Cancelar

</div>

</form>

</div>

</div>

9.3 Jogadores: listagem + filtro por turma

app/views/jogadores/index.php

php

Copiar

```
<div class="d-flex justify-content-between align-items-center
mb-3"> <h1 class="h3 mb-0">Jogadores</h1>

<a class="btn btn-success" href="/jogadores/create">Novo jogador</a>

</div>

<div class="card shadow-sm mb-3">

<div class="card-body">

<form class="row g-2 align-items-end" method="get" action="/jogadores">

<div class="col-md-4">

<label class="form-label">Filtrar por turma</label>

<input class="form-control" name="turma" value="<?= htmlspecialchars($turma
?? ") ?>" placeholder="Ex.: 2A, 3B...">

</div>

<div class="col-md-3">

<button class="btn btn-outline-primary w-100">Filtrar</button>

</div>

<div class="col-md-3">
```

```
<a class="btn btn-outline-secondary w-100" href="/jogadores">Limpar</a>
```

42

Programação para Web III Introdução ao MVC

```
</div>
```

```
</form>
```

```
</div>
```

```
</div>
```

```
<div class="card shadow-sm">
```

```
<div class="card-body table-responsive">
```

```
<table class="table table-striped align-middle mb-0">
```

```
<thead>
```

```
<tr>
```

```
<th>Nome</th>
```

```
<th>Turma</th>
```

```
<th>Número</th>
```

```
</tr>
```

```
</thead>
```

```
<tbody>
```

```
<?php foreach ($jogadores as $j): ?>
```

```
<tr>
```

```
<td class="fw-semibold"><?= htmlspecialchars($j['nome']) ?></td>
```

```
<td><span class="badge text-bg-success"><?= htmlspecialchars($j['turma'])
```

```
?></span></td>
```

```
<td><?= $j['numero'] !== null ? (int)$j['numero'] : '-' ?></td> </tr>
```

```
<?php endforeach; ?>
```

```
</tbody>
```

```
</table>
```

```
</div>
```

```
</div>
```

9.4 Jogadores: cadastro

app/views/jogadores/create.php

php

Copiar

```
<h1 class="h3 mb-3">Novo jogador</h1>
```

```
<div class="card shadow-sm">
```

```
<div class="card-body">
```

```
<form method="post" action="/jogadores/create" class="row g-3">
```

```
<div class="col-md-6">
```

```
<label class="form-label">Nome</label>
```

```
<input name="nome" class="form-control" required>
```

```
</div>
```

```
<div class="col-md-3">
```

```
<label class="form-label">Turma</label>
```

```
<input name="turma" class="form-control" placeholder="Ex.: 2A" required>
```

44

Programação para Web III Introdução ao MVC

```
</div>
```

```
<div class="col-md-3">
```

```
<label class="form-label">Número (opcional)</label>
```

```
<input name="numero" type="number" class="form-control" min="0">
```

```
</div>
```

```
<div class="col-12">
```

```
<button class="btn btn-success">Salvar</button>
```

```
<a class="btn btn-outline-secondary" href="/jogadores">Cancelar</a>
```

```
</div>
```

```
</form>
```

```
</div>
```

```
</div>
```

10) Partidas “mais completas”: formulário com Bootstrap + cartões no detalhe

10.1 Partidas: cadastro (Bootstrap + select de

modalidade) app/views/partidas/create.php

php

Copiar

```
<h1 class="h3 mb-3">Cadastrar partida</h1>
```

45

Programação para Web III Introdução ao MVC

```
<div class="card shadow-sm">
```

```
<div class="card-body">
```

```
<form method="post" action="/partidas/create" class="row g-3">
```

```
<div class="col-md-4">
```

```
<label class="form-label">Modalidade</label>
```

```
<select name="modalidade_id" class="form-select" required>
```

```
<option value="">Selecione...</option>
```

```
<?php foreach ($modalidades as $m): ?>
```

```
<option value="<?= (int)$m['id'] ?>"><?= htmlspecialchars($m['nome'])
```

```
?></option>
```

```
<?php endforeach; ?>
```

```
</select>
```

```
</div>
```

```
<div class="col-md-4">
```

```
<label class="form-label">Data e hora</label>
```

```
<input type="datetime-local" name="data_hora" class="form-control" required>
```

```
</div>
```

```
<div class="col-md-2">
```

```
<label class="form-label">Time casa</label>
```

```
<input name="time_casa" class="form-control" placeholder="Ex.: 2A" required>
```

46

Programação para Web III Introdução ao MVC

```
</div>
```

```
<div class="col-md-2">
```

```
<label class="form-label">Time fora</label>
```

```
<input name="time_fora" class="form-control" placeholder="Ex.: 3B" required>
```

```
</div>
```

```
<div class="col-12">
```

```
<button class="btn btn-success">Salvar</button>
```

```
<a class="btn btn-outline-secondary" href="/partidas">Cancelar</a>
```

```
</div>
```

```
</form>
```

```
</div>
```

```
</div>
```

10.2 Partidas: detalhe + registrar cartão na própria tela

Para isso, o `PartidaController@show` precisa também carregar jogadores. Um jeito simples (didático) é buscar todos.

Ajuste no `PartidaController@show()` (adicione antes do render):

php

Copiar

```
$pdo = Database::connection();
```

47

Programação para Web III Introdução ao MVC

```
$jogadores = $pdo->query("SELECT id, nome, turma, numero FROM jogadores  
ORDER BY turma, nome")->fetchAll();
```

E mande para a view (se você estiver usando render):

```
compact('partida','cartoes','jogadores').
```

app/views/partidas/show.php (trecho do formulário de cartão)

php

Copiar

```
<div class="d-flex justify-content-between align-items-start
```

```
mb-3"> <div>
```

```
<h1 class="h3 mb-1"><?= htmlspecialchars($partida['modalidade']) ?></h1>
```

```
<div class="text-muted">
```

```
<?= htmlspecialchars($partida['time_casa']) ?> x <?=  
htmlspecialchars($partida['time_fora']) ?>
```

```
• <?= htmlspecialchars($partida['data_hora']) ?>
```

```
</div>
```

```
</div>
```

```
<a class="btn btn-outline-secondary" href="/partidas">Voltar</a>
```

</div>

<div class="row g-3">

<div class="col-lg-6">

<div class="card shadow-sm">

48

Programação para Web III Introdução ao MVC

<div class="card-body">

<h2 class="h5">Placar</h2>

<form method="post" action="/partidas/placar" class="row g-2 align-items end">

<input type="hidden" name="id" value="<?= (int)\$partida['id'] ?>">

<div class="col-6">

<label class="form-label">Casa</label>

<input type="number" name="placar_casa" class="form-control" min="0" value="<?= (int)\$partida['placar_casa'] ?>">

</div>

<div class="col-6">

<label class="form-label">Fora</label>

<input type="number" name="placar_fora" class="form-control" min="0" value="<?= (int)\$partida['placar_fora'] ?>">

</div>

```
<div class="col-12">
```

```
<button class="btn btn-primary w-100">Atualizar placar</button>
```

```
</div>
```

```
</form>
```

```
</div>
```

```
</div>
```

49

Programação para Web III Introdução ao MVC

```
</div>
```

```
<div class="col-lg-6">
```

```
<div class="card shadow-sm">
```

```
<div class="card-body">
```

```
<h2 class="h5">Registrar cartão</h2>
```

```
<form method="post" action="/cartoes/create" class="row g-2"> <input
```

```
type="hidden" name="partida_id" value="<?= (int)$partida['id'] ?>">
```

```
<div class="col-12">
```

```
<label class="form-label">Jogador</label>
```

```
<select name="jogador_id" class="form-select" required>
```

```
<option value="">Selecione...</option>
```

```
<?php foreach ($jogadores as $j): ?>
```

```
<option value="<?= (int)$j['id'] ?>">
```

```
<?= htmlspecialchars($j['turma']) ?> — <?= htmlspecialchars($j['nome']) ?>
```

```
<?= $j['numero'] !== null ? ('#' . (int)$j['numero'] . ')') : " ?>
```

```
</option>
```

```
<?php endforeach; ?>
```

```
</select>
```

```
</div>
```

```
<div class="col-6">
```

```
<label class="form-label">Tipo</label>
```

```
<select name="tipo" class="form-select" required>
```

```
<option value="amarelo">Amarelo</option>
```

```
<option value="vermelho">Vermelho</option>
```

```
</select>
```

```
</div>
```

```
<div class="col-6">
```

```
<label class="form-label">Minuto (opcional)</label>
```

```
<input type="number" name="minuto" class="form-control" min="0">
```

```
</div>
```

```
<div class="col-12">
```

```
<label class="form-label">Observação (opcional)</label>
```

```
<input name="observacao" class="form-control" placeholder="Ex.: falta, reclamação...">
```

```
</div>
```

```
<div class="col-12">
```

```
<button class="btn btn-warning w-100">Salvar cartão</button>
```

```
</div>
```

54

Programação para Web III Introdução ao MVC

```
</form>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<div class="col-12">
```

```
<div class="card shadow-sm">
```

```
<div class="card-body">
```

```
<h2 class="h5">Cartões registrados</h2>
```

```
<div class="table-responsive">
```

```
<table class="table table-sm align-middle mb-0">
```

```
<thead>
```

```
<tr>
```

<th>Tipo</th>

<th>Jogador</th>

<th>Turma</th>

<th>Minuto</th>

<th>Obs.</th>

</tr>

</thead>

<tbody>

<?php foreach (\$cartoes as \$c): ?>

52

Programação para Web III Introdução ao MVC

<tr>

<td>

<span class="badge <?= \$c['tipo'] === 'vermelho' ? 'text-bg-danger' :
'text-bg-warning' ?>">

<?= htmlspecialchars(\$c['tipo']) ?>

</td>

<td class="fw-semibold"><?= htmlspecialchars(\$c['jogador']) ?></td> <td><?=
htmlspecialchars(\$c['turma']) ?></td>

<td><?= \$c['minuto'] !== null ? (int)\$c['minuto'] . "" : '-' ?></td> <td><?=
htmlspecialchars(\$c['observacao']) ?? '-' ?></td> </tr>

<?php endforeach; ?>

<?php if (empty(\$cartoes)): ?>

```
<tr><td colspan="5" class="text-muted">Nenhum cartão registrado.</td></tr>
```

```
<?php endif; ?>
```

```
</tbody>
```

```
</table>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

11) Próximos “degraus” para ficar realmente completo (sem virar caos)

Agora o sistema já tem “cara de produto”. Para completar o Interclasse com regras reais, os próximos passos mais naturais são:

- **Times (por turma) + elenco por modalidade**

- Tabela times (turma, modalidade)
- Tabela time_jogadores (time_id, jogador_id)

- **Partida com súmula**

- gols/pontos por jogador (e não só placar final)

- **Classificação automática**

- pontos: vitória=3, empate=1, derrota=0 (para modalidades com empate)
- saldo, gols pró/contra

- Segurança:

- **CSRF token** nos formulários (principalmente em produção)
- validação mais rigorosa (ex.: minuto máximo, jogador pertencente aos times da partida)

A grande sacada pedagógica aqui: **quando o domínio cresce (interclasse “de verdade”), o MVC impede que o projeto vire uma gelatina** — Models cuidam das regras/dados, Controllers orquestram, Views só exibem.